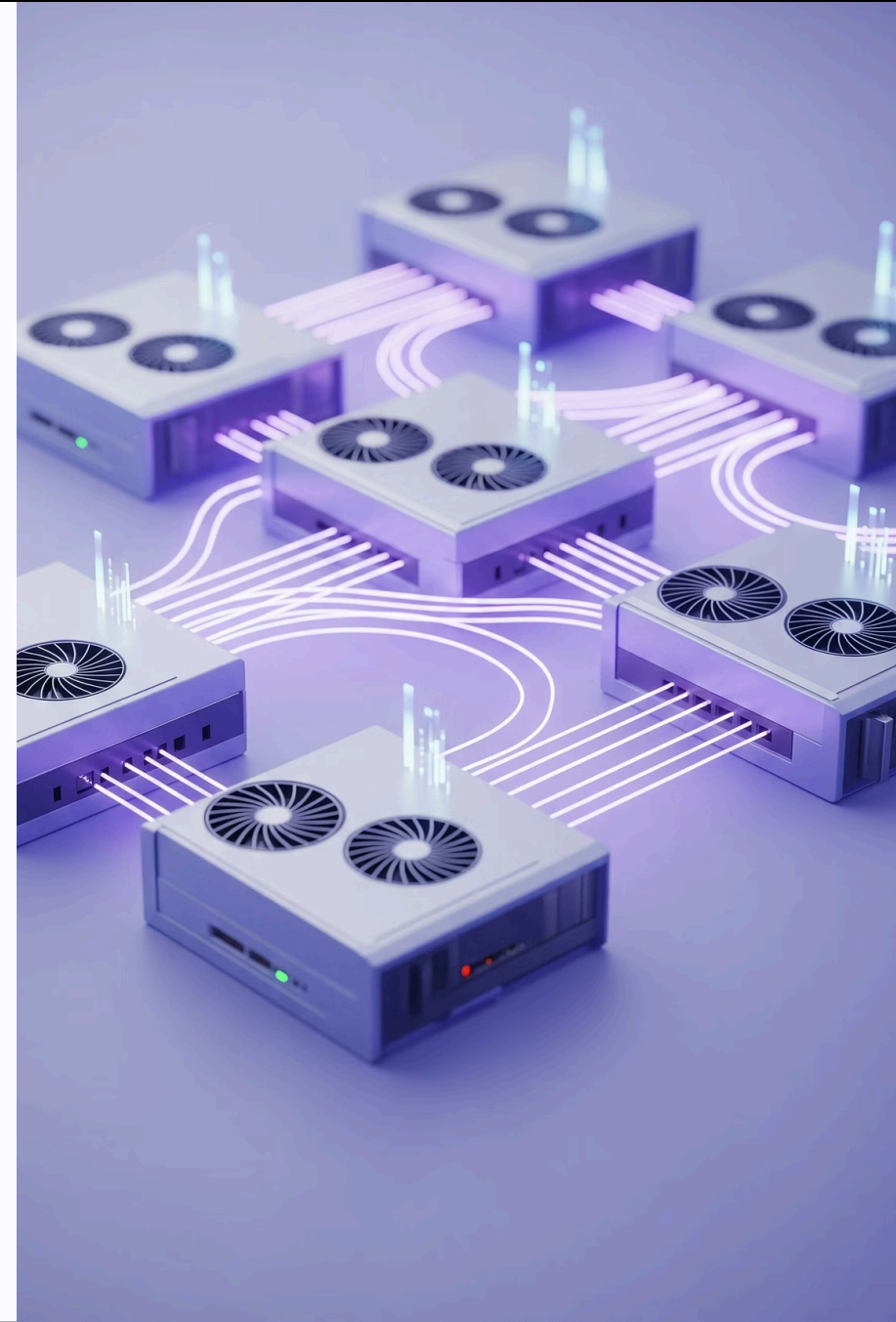


Fully Sharded Data Parallel (FSDP)

PyTorch's Native Memory-Sharding Engine

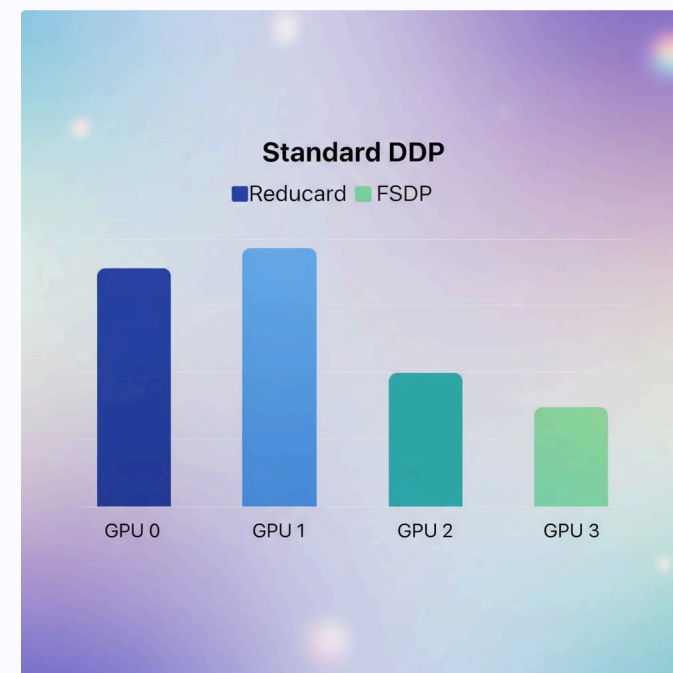
Enabling efficient training of billion-parameter models through innovative memory optimization techniques built directly into the PyTorch framework.



Why FSDP?

As model sizes continue to grow exponentially, standard Distributed Data Parallel (DDP) approaches hit hard memory limits. FSDP solves this by:

- Bringing ZeRO-style memory sharding **natively** into PyTorch
- Intelligently sharding **parameters, gradients, and optimizer states**
- Enabling training of **100B+ parameter models** on modest hardware



Core Concept: Memory Sharding

Distributed Storage

Each GPU stores only **a shard of parameters/states** rather than full copies

Just-in-Time Assembly

During forward pass, layer parameters are gathered precisely when needed

Efficient Gradient Flow

During backward pass, gradients undergo reduce-scatter operations

Near-Linear Scaling

Memory savings scale almost linearly with the number of GPUs used



Key Features

Full-State Sharding

Distributes parameters, gradients, and optimizer states across devices

PyTorch Integration

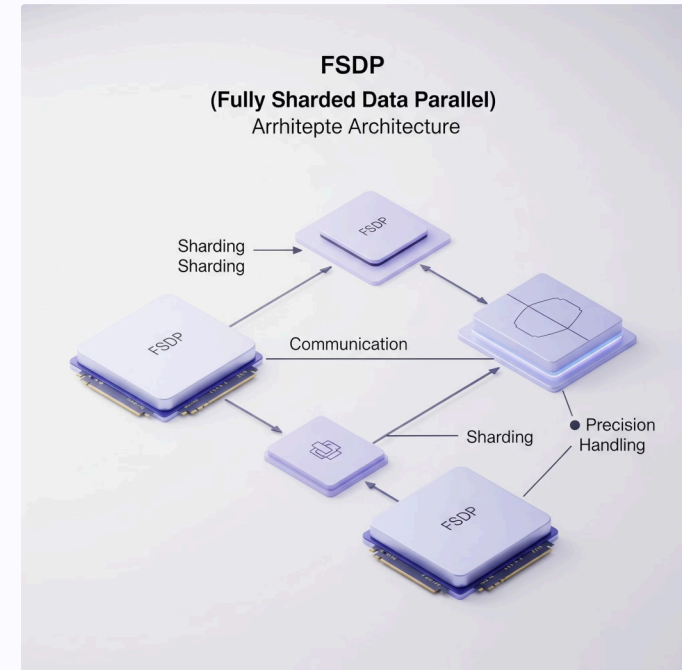
Built directly into PyTorch core (`torch.distributed.fsdp`)

Precision Options

Supports mixed precision training (FP16/BF16)

Memory Extensions

Works with activation checkpointing, CPU/NVMe offload



FSDP vs ZeRO: Comparison

Core Similarities

- Both shard model states across multiple GPUs
- Both enable training larger models with limited hardware
- Both follow similar communication patterns

Key Differences

- **ZeRO (DeepSpeed)**: External framework, more mature offload features
- **FSDP**: Native PyTorch implementation, simpler integration, tighter ecosystem fit

FSDP continues to gain feature parity while maintaining the benefits of native PyTorch implementation.

Example Usage

```
import torch
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP
)

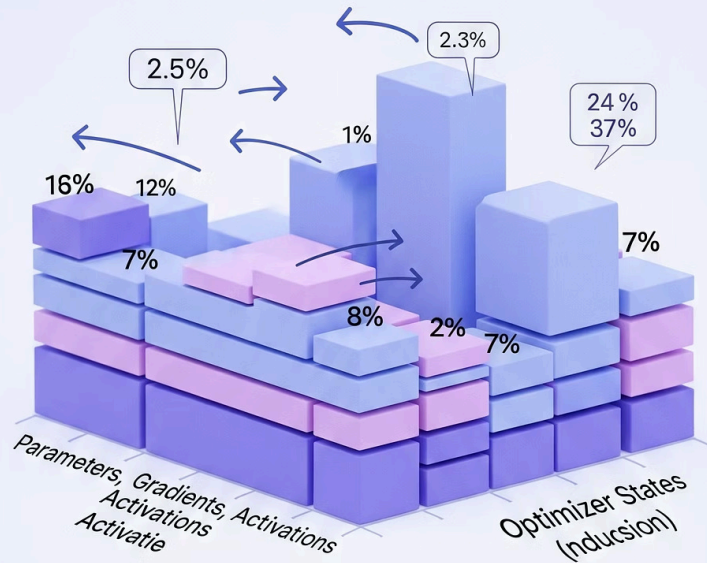
model = MyTransformer()
sharded_model = FSDP(model) # wrap layers or full model
loss_fn = torch.nn.CrossEntropyLoss()

# Train as usual
for inputs, targets in dataloader:
    outputs = sharded_model(inputs)
    loss = loss_fn(outputs, targets)
    loss.backward()
    optimizer.step()
```

Implementation Notes

- Wrap per-layer for finer granularity control
- Launch with torchrun: `torchrun --nproc_per_node=8 train.py`
- Auto-wrapping policies available for complex models
- Minimal code changes from standard PyTorch

Memory Usage



Parameters Gradients, Activations
Activations
Optimizer States
Quantization
Sparse Matrices
Optimizer

Memory Optimizations

1 Activation Checkpointing

Recompute activations during backward pass instead of storing them, trading compute for memory

2 Mixed Precision

Use FP16/BF16 for parameters, activations, and communications to reduce memory footprint by up to 2x

3 CPU Offload

Move parameter shards to CPU between computation steps, freeing GPU memory for larger batch sizes

4 Auto-Wrap Policies

Selectively shard only the largest layers while keeping small layers replicated for better performance



Performance Considerations

When to Use FSDP

- Best for **large Transformer-style models** (10B+ parameters)
- When memory is the primary constraint
- Multi-node training setups
- When you need to increase batch size

Tuning for Performance

- Adjust communication bucket sizes
- Overlap communication with computation
- Benchmark **tokens/sec/GPU** before/after enabling
- Small models may actually **slow down** from overhead

Fault Tolerance

Sharded Checkpoints

Each GPU saves only its parameter shards, dramatically reducing disk footprint for large models

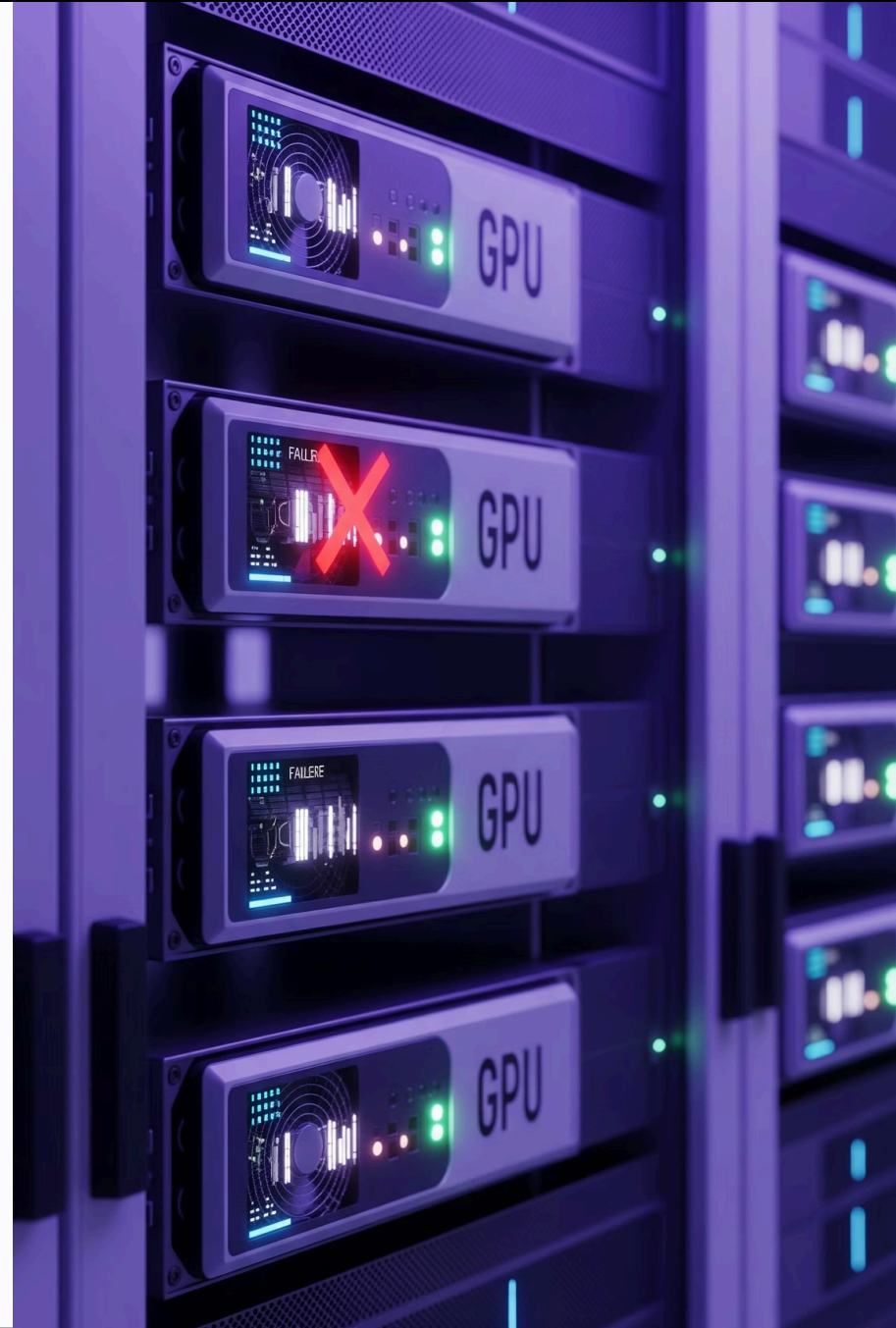
Elastic Training

Supports dynamic worker count changes via TorchElastic integration with Slurm/Kubernetes

Recovery Testing

Critical to validate recovery processes to prevent silent data corruption during long training runs

FSDP's checkpoint mechanism means even 100B+ parameter models can be saved efficiently without massive storage requirements.



Best Practices



Strategic Wrapping

Use auto-wrap to shard Transformer blocks, not tiny layers - find the right granularity



BF16 Precision

Start with BF16 mixed precision when available - better numerical stability than FP16



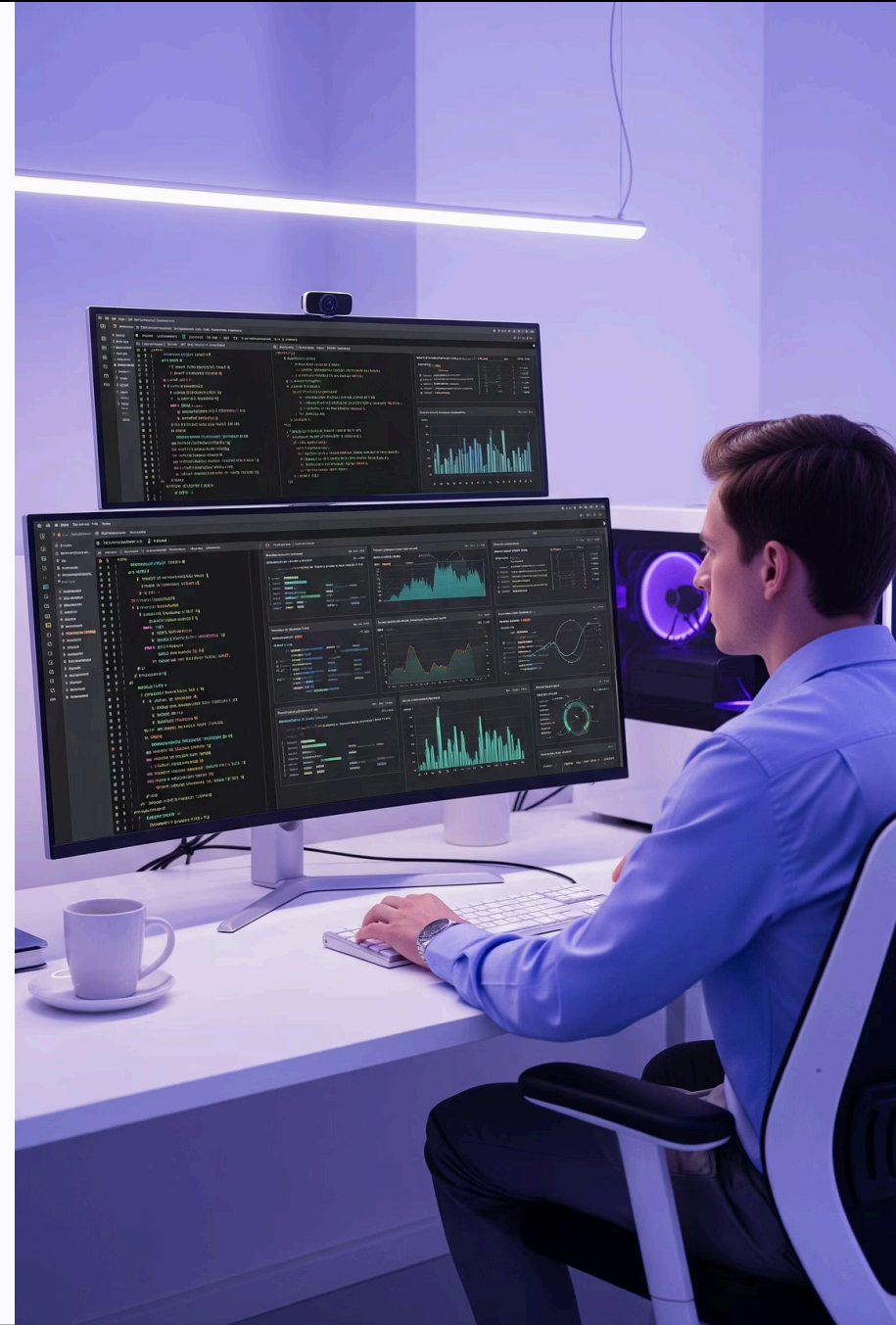
Gradient Accumulation

Use gradient accumulation for stable global batch sizes across distributed setups



Thorough Benchmarking

Always compare against DDP before switching production workloads to FSDP



Key Takeaways



Native Integration

FSDP brings **ZeRO-style sharding** directly into the PyTorch ecosystem



Memory Efficiency

Dramatically reduces memory requirements by distributing all model states



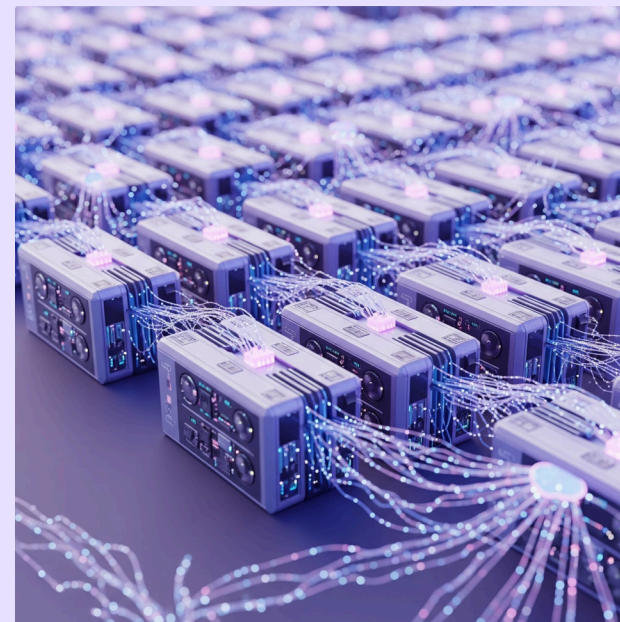
Ideal Use Case

Perfect for multi-node Transformer & LLM training at scale



Scale Enabler

Key technology for training **100B+ parameter models** within PyTorch



Start using FSDP today for your next large-scale training project!